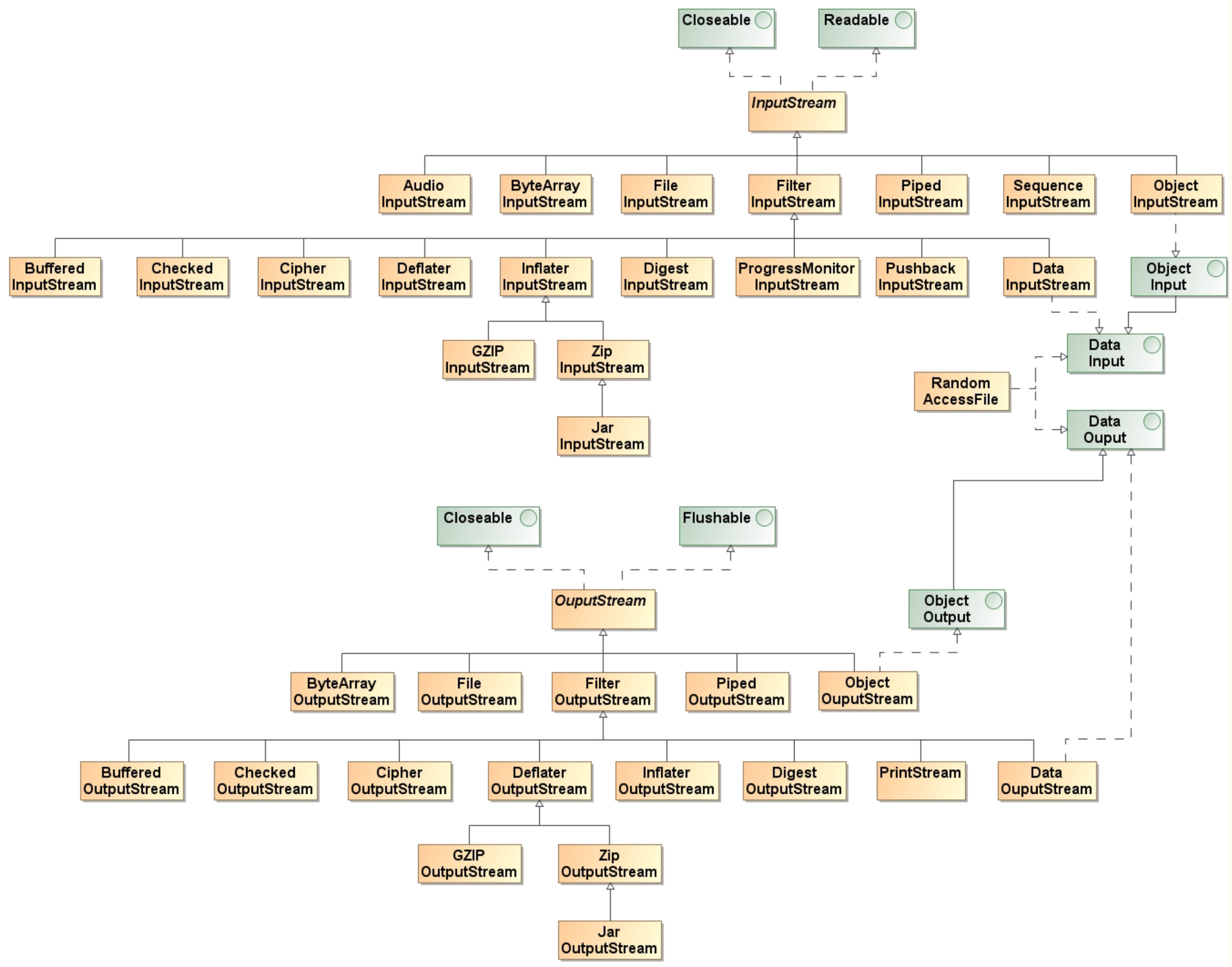
Flujos y Ficheros

1. Flujos
2. Flujos de Bytes
3. Flujos de Caracteres
4. Flujos de Utilidades
5. Flujos de Datos
6. Ficheros
7. Ficheros de Texto
8. Ficheros Binarios
9. Serialización de Objetos
10. Flujos de Objetos

FLUJOS: los flujos (streams)son objetos que representan secuencias ordenadas de datos que tienen una fuente (flujos de entrada) o un destino (flujos de salida);

- Las clases de entrada/salida del paquete java.io se definen en términos de flujos, y permiten a los programadores abstraerse de los detalles específicos del sistema operativo al acceder a los recursos del sistema, tales como ficheros, pantalla, teclado, líneas de comunicaciones, etc.;
- En la biblioteca se distinguen dos grandes tipos de flujos:
  - de entrada y salida de bytes, cuyas clases abstractas de las que heredan todas las demás son **InputStream** y **OutputStream**;
  - de entrada y salida de caracteres, cuyas clases abstractas de las que heredan todas las demás son **Reader** y **Writer**;

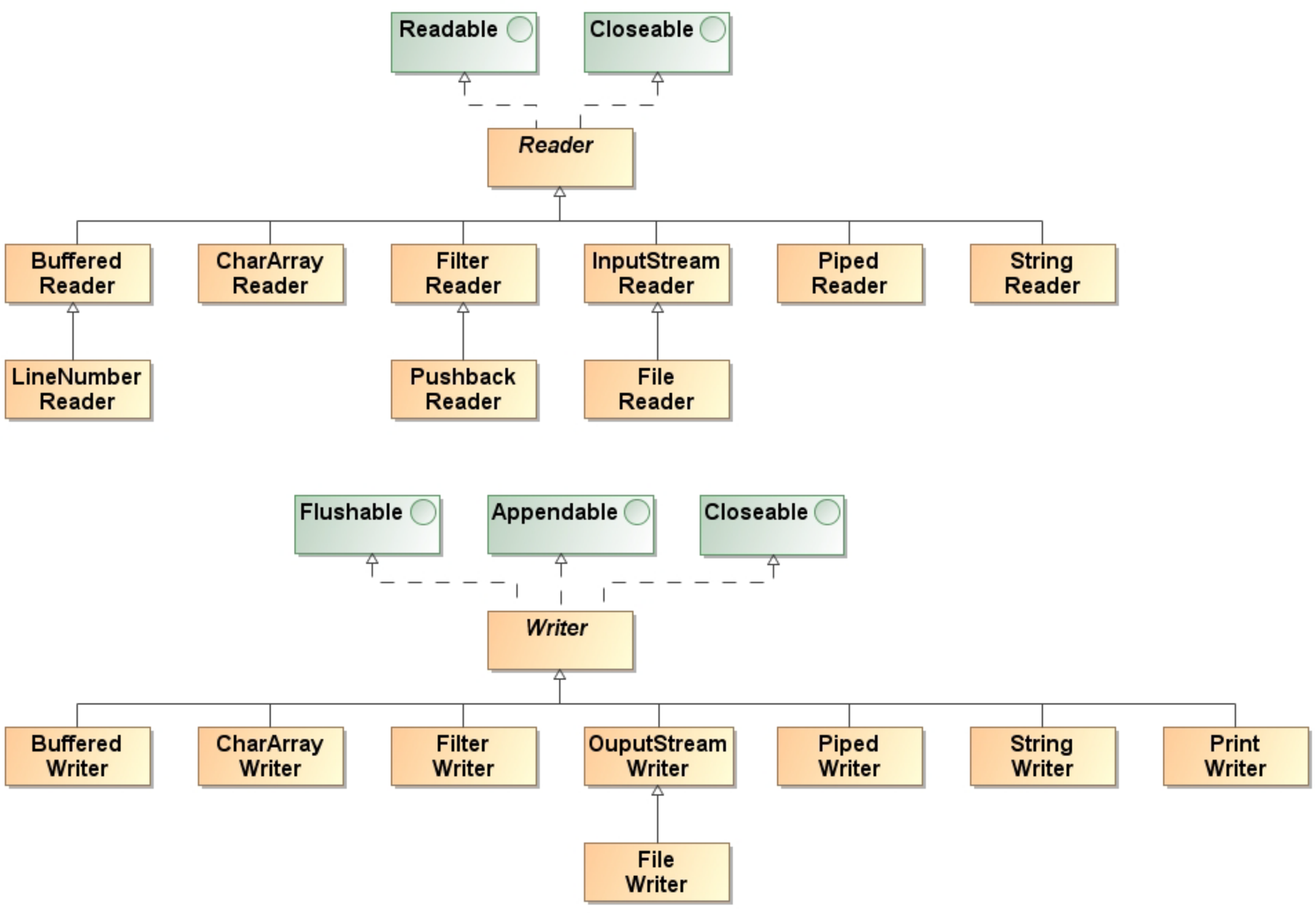

Closeable
Readable
InputStream
Audio InputStream
ByteArray InputStream
File InputStream
Filter InputStream
Piped InputStream
Sequence InputStream
Object InputStream
Buffered InputStream
Checked InputStream
Cipher InputStream
Deflater InputStream
Inflater InputStream
Digest InputStream
ProgressMonitor InputStream
Pushback InputStream
Data InputStream
Object Input
GZIP InputStream
Zip InputStream
Jar InputStream
Data Input
Random AccessFile
Data Ouput
Closeable
Flushable
OuputStream
Object Output
ByteArray OutputStream
File OutputStream
Filter OutputStream
Piped OutputStream
Object OuputStream
Buffered OutputStream
Checked OutputStream
Cipher OutputStream
Deflater OutputStream
Inflater OutputStream
Digest OutputStream
PrintStream
Data OuputStream
GZIP OutputStream
Zip OutputStream
Jar OutputStream

FLUJOS DE BYTES: Clase InputStream:

public abstract int **read**() throws IOException
public int **read**(byte[] b) throws IOException
public int **read**(byte[] b, int off, int len) throws IOException
public long **skip**(long n) throws IOException
public int **available**() throws IOException
public void close() throws IOException

FLUJOS DE BYTES: Clase OutputStream:

public abstract void **write**(int b) throws IOException
public void **write**(byte[] b) throws IOException
public void **write**(byte[] b, int off, int len) throws IOException
public void **flush**() throws IOException
public void **close**() throws IOException


Readable
Closeable
Reader
Buffered Reader
CharArray Reader
Filter Reader
InputStream Reader
Piped Reader
String Reader
LineNumber Reader
Pushback Reader
File Reader
Flushable
Appendable
Closeable
Writer
Buffered Writer
CharArray Writer
Filter Writer
OuputStream Writer
Piped Writer
String Writer
Print Writer
File Writer

FLUJOS DE CARACTERES: Clase Reader:

public int **read**() throws IOException
public int **read**(char[] cbuf) throws IOException
public abstract int **read**(char[] cbuf, int off, int len) throws IOException
public long **skip**(long n) throws IOException
public boolean **ready**() throws IOException
public abstract void **close**() throws IOException

FLUJOS DE CARACTERES: Clase Writer:

public void **write**(int c) throws IOException
public void **write**(char[] cbuf) throws IOException
public abstract void **write**(char[] cbuf, int off, int len) throws IOException
public void **write**(String str) throws IOException
public void **write**(String str, int off, int len) throws IOException
public Writer **append**(char c) throws IOException
public Writer **append**(CharSequence csq) throws IOException
public Writer **append**(CharSequence csq, int start, int end) throws IOException
public abstract void **flush**() throws IOException
public abstract void **close**() throws IOException

FLUJOS DE CARACTERES: Flujos de conversión:

- Las clases InputStreamReader y OuputStreamWriter permiten convertir bytes en caracteres y caracteres en bytes, respectivamente;
- Los objetos InputStreamReader reciben en su constructor un objeto InputStream que es capaz de leer bytes;

public **InputStreamReader**(InputStream in)

- Los métodos read de InputStreamReader leen bytes de su InputStream y los convierten en caracteres;
- Los objetos OutputStreamReader reciben en su constructor un objeto OutputStream que es capaz de escribir bytes;

public **OuputStreamWriter**(OutputStream out)

- Los métodos write de OutputStreamReader convierten caracteres a bytes y los escriben en su OutputStream;

FLUJOS DE BYTES Y DE CARACTERES: Flujos de Utilidades (I):

- hay varios tipos de flujos que definen comportamientos específicos, tanto en su modalidad de entrada como de salida:
- los flujos de tipo *Filter* aplican alguna operación de filtrado cuando leen o escriben datos utilizando otro flujo, por ejemplo, un objeto *FilterReader* lee caracteres de otro objeto *Reader*, procesa los caracteres de alguna manera, y devuelve el resultado;

protected **FilterReader**(Reader in)

- los flujos de tipo *Buffered* añaden almacenamiento intermedio en forma de *buffers*, de forma que las lecturas y escrituras no requieren acceder al sistema de ficheros en cada invocación. Los flujos de caracteres de este tipo también proporcionan la lectura y escritura de líneas de texto (con retornos de línea);

public **BufferedReader**(Reader in)

public **BufferedWriter**(Writer out)

FLUJOS DE BYTES Y DE CARACTERES: Flujos de Utilidades (II):

- los flujos de tipo *Piped* se diseñan en parejas, de forma que las escrituras que realiza un flujo de salida se pueden leer en otro flujo de entrada;

public **PipedWriter**(PipedReader snk) throws IOException

- los llamados flujos en memoria permiten usar estructuras de datos en memoria como fuente o destino de un flujo:
  - los flujos *ByteArray* usan un vector de byte,
  - los flujos *CharArray* usan un vector de char,
  - los flujos *String* usan objetos de la clase String;
- los flujos de tipo *Print* proporcionan los métodos *print* y *println* que facilitan la escritura de valores de tipos primitivos y objetos en forma de texto legible (*System.out* es un flujo *PrintStream*), además, todos los flujos *Print* actúan como flujos *Filter*;

FLUJOS DE BYTES: Flujos de Datos:

- leer y escribir bytes y caracteres de texto es muy útil, pero a menudo se necesita transmitir datos binarios de los tipos primitivos a través de un flujo;
- Los interfaces *DataInput* y *DataOutput* declaran métodos que permiten leer y escribir datos en formato binario:

Tipo	Lectura	Escritura
boolean	boolean readBoolean()	writeBoolean(boolean)
char	char readChar()	writeChar(char)
short	short readShort()	writeShort(short)
int	int readInt()	writeInt(int)
long	long readLong()	writeLong(long)
float	float readFloat()	writeFloat(float)
double	double readDouble()	writeDouble(double)
String	String readUTF()	writeUTF(String)

FICHEROS:

- los flujos *File (FileInputStream, FileOutputStream, FileReader* y *FileWriter)* permiten tratar a un fichero como un flujo de entrada o de salida;
- Para crear un objeto de este tipo de flujos, se usa un constructor que recibe como argumento el nombre del fichero;

Ej.

```
FileWriter outTxt = new FileWriter("salida.txt");
FileReader inTxt = new FileReader("entrada.txt");
FileOutputStream outDat = new FileOutputStream("salida.dat");
FileInputStream inDat= new FileInputStream("entrada.dat");
```

ESCRITURA EN FICHEROS DE TEXTO:

- para escribir en un fichero de texto se usa la clase PrintWriter;

Ej.

```
PrintWriter out = null;
try {
	out = new PrintWriter("fichero.txt");
	out.print(1);
	out.print(':');
	out.print(" juan = ");
	out.println(5.5);
	out.println(2 + ": jose = " + 7.2);
} catch (IOException ex){
	System.out.println("IOExceptional escribir:" + ex.getMessage());
} finally{
	if (out != null) {
		out.close();
	}
}
```

LECTURA DE FICHEROS DE TEXTO:

•para leer de un fichero de texto se usa la clase BufferedReader;

```
BufferedReader in = null;
try{
	in = new BufferedReader(new FileReader("fichero.txt"));
	String linea;
	while((linea= in.readLine()) != null) {
		System.out.println(linea);
	}
} catch (IOException ex){
	System.out.println("IOExceptional leer: " + ex.getMessage());
} finally{
	if(in != null) {
	try{
		in.close();
	} catch (IOException ex){
		System.out.println("IOExceptional cerrar: " + ex.getMessage());
	}
}
}
```

ESCRITURA EN FICHEROS BINARIOS:

•para escribir en un fichero binario se usa la clase DataOutputStream;

```
DataOutputStream out= null;
try{
    out = new DataOutputStream(new FileOutputStream("fichero.dat"));
    out.writeInt(1);
    out.writeChar(':');
    out.writeUTF(" juan= ");
    out.writeDouble(5.5);
} catch (IOException ex) {
    System.out.println("IOExceptional escribir: " + ex.getMessage());
} finally{
    if(out!= null) {
        try{
            out.close();
        } catch (IOException ex) {
            System.out.println("IOExceptional cerrar: " +
                ex.getMessage());
        }
    }
}
```

LECTURA DE FICHEROS BINARIOS:

- para leer de un fichero binario se usa la clase DataInputStream;

```
DataInputStream in = null;
try{
    in = new DataInputStream(new FileInputStream("fichero.dat"));
    System.out.println(in.readInt()+ String.valueOf(in.readChar()) +
    in.readUTF()+ in.readDouble());
    System.out.println(in.readInt() + String.valueOf(in.readChar()) +
    in.readUTF()+ in.readDouble());
} catch (IOException ex) {
    System.out.println("IOExceptional leer: " + ex.getMessage());
} finally{
    if(in != null) {
        try{
            in.close();
        } catch (IOException ex) {
            System.out.println("IOExceptional cerrar: " + ex.getMessage());
        }
    }
}
```

SERIALIZACIÓN DE OBJETOS

Resuelve el problema de almacenar en un fichero (o transmitir por una línea de comunicación, etc.) un conjunto de objetos de distintas clases, posiblemente relacionados entre sí, y posteriormente restaurarlos de nuevo en la memoria de un programa, de forma que se mantengan el mismo número de objetos, los mismos valores de sus atributos y las mismas relaciones entre ese conjunto de objetos que en la situación original;

Ej.

```
public abstract class Persona {
    private String nombre;
    protected Persona() {
        nombre = "<vacio>";
    }
    public Persona(String nombre) {
        this.nombre= nombre;
    }
    protected String getNombre() {
        return nombre;
    }
    public abstract void escribir();
}
```

```
public class Profesor extends Persona {
    private int codigo;
    private Alumno[] alumnos = new Alumno[100];
    private int numAlumnos= 0;
    public Profesor(String nombre, int codigo) {
        super(nombre);
        this.codigo= codigo;
    }
    public void setAlumno(Alumno alumno) {
        alumnos[numAlumnos++] = alumno;
        alumno.setProfesor(this);
    }
}

public class Alumno extends Persona {
    private float nota = 0.1f;
    private Profesor[] profesores = new Profesor[100];
    private int numProfesores= 0;
    public Alumno(String nombre) {
        super(nombre);
    }
    public void setProfesor(Profesor profesor) {
        profesores[numProfesores++] = profesor;
    }
}
```

```
public class Escuela {
    privateProfesor[] profesores = new Profesor[100];
    private int numProfesores= 0;
    private Alumno[] alumnos = new Alumno[100];
    private int numAlumnos= 0;
    publicvoid iniciar() {
        Alumno jose= new Alumno("jose");
        Alumno juan= new Alumno("juan");
        Alumno maria= new Alumno("maria");
        Profesor ana= new Profesor("ana", 1);
        ana.setAlumno(jose);
        ana.setAlumno(juan);
        Profesor jesus= new Profesor("jesus", 2);
        jesus.setAlumno(juan);
        jesus.setAlumno(maria);
        personas[numPersonas++] = ana;
        personas[numPersonas++] = jesus;
        personas[numPersonas++] = jose;
        personas[numPersonas++] = juan;
        personas[numPersonas++] = maria;
        ...
        this.guardar();
        ...
        this.borrar();
        ...
        this.restaurar();
        ...
        }
```

SERIALIZACIÓN DE OBJETOS

- resolver este problema mediante ficheros de texto o binarios es complejo:
  - porque los objetos tienen un tamaño variable dependiendo de su clase,
  - porque los datos de los objetos deberían almacenarse una sola vez en el fichero,
  - porque no se pueden almacenar y restaurar las referencias, ya que se corresponden con direcciones de memoria, y al volver a cargar un objeto, éste ocupará una dirección de memoria diferente de la original;

SERIALIZACIÓN DE OBJETOS

- la solución consiste en guardar en el fichero cada objeto con un “número de serie” único;
- al almacenar un objeto en el fichero:
    - se asocia un número de serie a cada referencia,
    - al encontrar por primera vez una referencia, se almacenan los datos del objeto en el fichero y su número de serie,
    - las siguientes veces que se encuentre esa referencia, escribir sólo el número de serie;
- al restaurar un objeto del fichero:
    - al restaurar por primera vez un objeto, se crea, se inicializa con sus datos, y se recuerda la asociación entre la nueva referencia y su número de serie,
    - al encontrar sólo su número de serie, recuperar la referencia de ese número de serie;

FLUJOS DE OBJETOS:

Java proporciona los flujos de bytes *ObjectOutputStream* y *ObjectInputStream* para resolver automáticamente este problema;

- para almacenar (**serializar**) los objetos se usa la clase ObjectOutputStream, que implementa el interfaz ObjectOutput que, a su vez, extiende el interfaz DataOutput, con lo que, además de poder escribir cualquier dato en formato binario, añade entre otros los siguientes métodos:

  public **ObjectOutputStream**(OutputStream out) throws IOException
  public final void **writeObject**(Object obj) throws IOException
  public void **defaultWriteObject**() throws IOException

- que permiten serializar objetos en un flujo de bytes;

FLUJOS DE OBJETOS:

•para restaurar (*deserializar*) los objetos se usa la clase *ObjectInputStream*, que implementa el interfaz *ObjectInput* que, a su vez, extiende el interfaz *DataInput*, con lo que, además de poder leer cualquier dato en formato binario, añade entre otros los siguientes métodos:

public **ObjectInputStream**(InputStreamin) throws IOException
public final Object **readObject()** throws IOException, ClassNotFoundException
public void **defaultReadObject()** throws IOException, ClassNotFoundException

•que permiten deserializar objetos desde un flujo de bytes;

FLUJOS DE OBJETOS:

•el único requisito que se necesita para poder almacenar y restaurar objetos en los flujos de objetos es que las clases de esos objetos deben implementar el interfaz *java.io.Serializable*:

```
class Clase implementsSerializable{ ... }
```

•el interfaz *Serializable* no tiene ningún método, por lo que, en principio, no hay que modificar nada más en las clases de los objetos a serializar;
•el mecanismo de serialización por defecto de Java se encarga de almacenar y restaurar automáticamente todos los atributos de los objetos, excepto los atributos *static*; que nunca se serializan ni se deserializan;

SERIALIZACIÓN POR DEFECTO:

•en la serialización por defecto, todos los atributos que sean tipos primitivos o cadenas de texto se escriben con el formato usado en *DataOutputStream*;

•en la serialización por defecto, todos los campos que sean referencias a objetos deben ser, a su vez, serializables;

•si durante el proceso de serialización o deserialización se intenta almacenar o restaurar algún objeto no serializable, se eleva una excepción de la clase *NotSerializableException*;

SERIALIZACIÓN POR DEFECTO:

•si durante la deserialización no se encuentra la clase de un objeto o no se puede cargar por otra razón, se eleva una excepción *ClassNotFoundException*;

•si una clase es una subclase de otra clase no serializable, la superclase debe tener un constructor sin argumentos que sea accesible desde la subclase, para que la deserialización por defecto pueda inicializar los atributos de las superclase, en caso contrario se produce una excepción *IOException*;

```
import java.io.Serializable;
public abstract class Persona implements Serializable{
	...
}
public class Escuela{
	private void guardar() {
		ObjectOutputStream out= null;
		try{
			out= new ObjectOutputStream(new
			FileOutputStream("fichero.dat"));
			out.writeInt(numPersonas);
			for(inti = 0; i < numPersonas; i++) {
				out.writeObject(personas[i]);
			}
		} catch (IOExceptionex) {
		System.out.println("IOExceptional escribir: " +
		ex.getMessage());
		} finally{
			if(out!= null) {
				try{
					out.close();
				} catch (IOExceptionex) {
				System.out.println("IOExceptional cerrar: " +
				ex.getMessage());
			}
		}
	}
}
```

```
private void restaurar() {
    personas = new Persona[100];
    numPersonas= 0;
    ObjectInputStream in = null;
    try{
        in = new ObjectInputStream(new FileInputStream("fichero.dat"));
        int total = in.readInt();
        for(inti = 0; i < total; i++) {
            personas[numPersonas] = (Persona) in.readObject();
            numPersonas++;
        }
    } catch (IOExceptionex) {
        System.out.println("IOExceptional leer: " +
        ex.getMessage());
    } catch (ClassNotFoundExceptionex) {
        System.out.println("ClassNotFoundExceptional leer: " +
        ex.getMessage());
    } finally{
        if(in != null) {
            try{
                in.close();
            } catch (IOExceptionex) {
            System.out.println("IOExceptional cerrar: " +
            ex.getMessage());
            }
        }
    }
}
```

SERIALIZACIÓN POR DEFECTO:

•otra solución más sencilla es escribir y leer directamente el vector en el fichero, en lugar de hacerlo elemento a elemento:

```
public class Escuela{
    private void guardar() {
        ObjectOutputStream out= null;
        try{
            out= new ObjectOutputStream(new
            FileOutputStream("fichero.dat"));
            out.writeInt(numPersonas);
            out.writeObject(personas);
        } catch (IOExceptionex) {
            System.out.println("IOExceptional escribir: " +
            ex.getMessage());
        } finally{
            if(out!= null) {
                try{
                    out.close();
                } catch (IOExceptionex) {
                    System.out.println("IOExceptional cerrar: " +
                    ex.getMessage());
                }
            }
        }
}
```

```
private void restaurar() {
    numPersonas= 0;
    ObjectInputStream in = null;
    try{
        in = new ObjectInputStream(new FileInputStream("fichero.dat"));
        int total = in.readInt();
        personas = (Persona[]) in.readObject();
        numPersonas= total;
    } catch (IOExceptionex) {
        System.out.println("IOExceptional leer: " + ex.getMessage());
    } catch (ClassNotFoundExceptionex) {
        System.out.println("ClassNotFoundExceptional leer: " +
        ex.getMessage());
    } finally{
        if(in != null) {
            try{
                in.close();
            } catch (IOExceptionex) {
                System.out.println("IOExceptional cerrar: " +
                ex.getMessage());
        }
    }
}
```

SERIALIZACIÓN PERSONALIZADA:

- en ciertas ocasiones es necesario personalizar el mecanismo de serialización por defecto de Java; por ejemplo:
  - ciertos atributos de los objetos de la clase no deben serializarse porque contienen información redundante o que no tiene sentido después del proceso (por ejemplo, enlaces de listas, referencias de ficheros abiertos, ventanas, etc.);
  - ciertos atributos de los objetos de la clase no implementan el interfaz Serializable, por lo que no deben serializarse mediante el mecanismo por defecto, sino que deben ser esos objetos los que deben responsabilizar almacenar y restaurar sus atributos ;
  - una clase es una subclase de otra clase no serializable, y también se debe responsabilizar de almacenar y restaurar sus atributos heredados;

SERIALIZACIÓN PERSONALIZADA:

•para indicar que algún atributo de una clase no debe ser serializado por el mecanismo por defecto, se añade a la declaración del atributo el modificador *transient*;

```
import pooa.util.Lista; // NO SERIALIZABLE
public class AlumnoDinamico extendsPersona {

    private transient Lista profesores = new Lista(); // NO SERIALIZABLE
    private int numProfesores= 0;
    ...
}
```

SERIALIZACIÓN PERSONALIZADA:

•cuando no se desea usar el mecanismo de serializaciónpor defecto para una clase, se deben definir los siguientes métodos, respetando exactamente sus cabeceras:

private void **readObject**(ObjectInputStream in)
    throws IOException, ClassNotFoundException
private void **writeObject**(ObjectOutputStream out)
    throws IOException

•en cuyo caso, los objetos de la clase no se serializan ni deserializan por el mecanismo por defecto, sino que:

- •en la serialización se invoca al método *writeObject* para que el objeto haga el proceso de la manera oportuna;
- •en la deserialización, sólo se crea el objeto de la clase, y se invoca el método *readObject* para que el objeto haga el proceso;

```
import pooa.util.Lista; // NO SERIALIZABLE
public class AlumnoDinamico extends Persona {

    private float nota = 0.1f;
    private transient Lista profesores = new Lista(); // NO SERIALIZABLE
    ...
    private void writeObject(ObjectOutputStream out) throws
    IOException{
        out.defaultWriteObject();
        out.writeInt(profesores.size());
        Iterador iterador= profesores.iterador();
        while(iterador.hasNext()) {
            Profesor profesor= (Profesor) iterador.next();
            out.writeObject(profesor);
        }
    }
    private void readObject(ObjectInputStream in) throwsIOException,
     ClassNotFoundException{
        in.defaultReadObject();
        profesores = new Lista();
        int numProfesores= in.readInt();
        for(inti = 0; i < numProfesores; i++) {
            Profesor profesor= (Profesor) in.readObject();
            profesores.add(profesor);
        }
    }
```